

MSAGLJS: Browsing Large Graphs with Tile Pyramids and Sleeve Routing

Anonymous author(s)

Anonymous affiliation(s)

Abstract

We present a new way to visualize a large graph in the style of online geographic maps. The method builds a *tile pyramid* for *semantic zoom*: at every zoom level the labels of the highest-ranked nodes remain readable, just as the names of major geographical features stay readable on those maps.

The edges are routed by a method we call *sleeve routing*, which computes for each edge the shortest path in its homotopy class around the node obstacles, using the Constrained Delaunay Triangulation. We apply several heuristics to speed up the routing.

We implemented our approach in the WebGL renderer of MSAGLJS, an open-source TypeScript library for graph visualization in web browsers, with the entire pipeline running client-side, without a dedicated server. We target graphs with at most 40K nodes and 100K edges. The renderer's demo can be seen at <https://microsoft.github.io/msagljs/renderer-webgl/index.html>.

MSAGLJS is available on GitHub¹ and as NPM packages².

2012 ACM Subject Classification Human-centered computing → Graph drawings; Theory of computation → Computational geometry

Keywords and phrases Graph visualization, edge routing, Constrained Delaunay Triangulation, tiling, WebGL

Category Track 2, Long Paper

Generative AI Declaration Generative AI was used for editorial assistance in the preparation of this article.

1 Introduction

Visualizing graphs in web browsers without a dedicated server presents unique challenges: the runtime is single-threaded, the per-tab JavaScript heap is capped at roughly 4 GB,³ and users expect the smooth pan-and-zoom experience familiar from online maps. MSAGLJS⁴ is an open-source TypeScript library that addresses these challenges by combining efficient layout algorithms, a novel edge routing method, and a tiling scheme for level-of-detail rendering.

MSAGLJS is consumed as a set of NPM packages and renders graphs using WebGL through the `deck.gl` framework [2]. It supports the layered Sugiyama scheme [44], Pivot MDS [14], and IPSep-CoLa [21], with edges routed around node obstacles. Throughout this paper our typical example is a social network laid out by IPSep-CoLa; the methods we describe, however, are agnostic to the layout algorithm and apply to any node placement in which the nodes do not overlap. The rendering pipeline builds a hierarchy of tiles—analogueous to map tiles—so that at any zoom level only a bounded number of entities are drawn. Tiles are delivered to the browser through the standard `TileLayer` of the `@deck.gl/geo-layers`

¹ <https://github.com/microsoft/msagljs>

² `@msagl/core`, `@msagl/drawing`, `@msagl/parser`, `@msagl/renderer-svg`, `@msagl/renderer-webgl` — all on <https://www.npmjs.com/>.

³ In current Chrome, V8 with pointer compression caps the per-tab JavaScript heap at approximately 4 GB (2³² bytes).

⁴ <https://github.com/microsoft/msagljs>



© Anonymous author(s);

licensed under Creative Commons License CC-BY 4.0

34th International Symposium on Graph Drawing and Network Visualization (GD 2026).

Editors: TBD; Article No. 0; pp. 0:1–0:14



Leibniz International Proceedings in Informatics

LIPIC Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

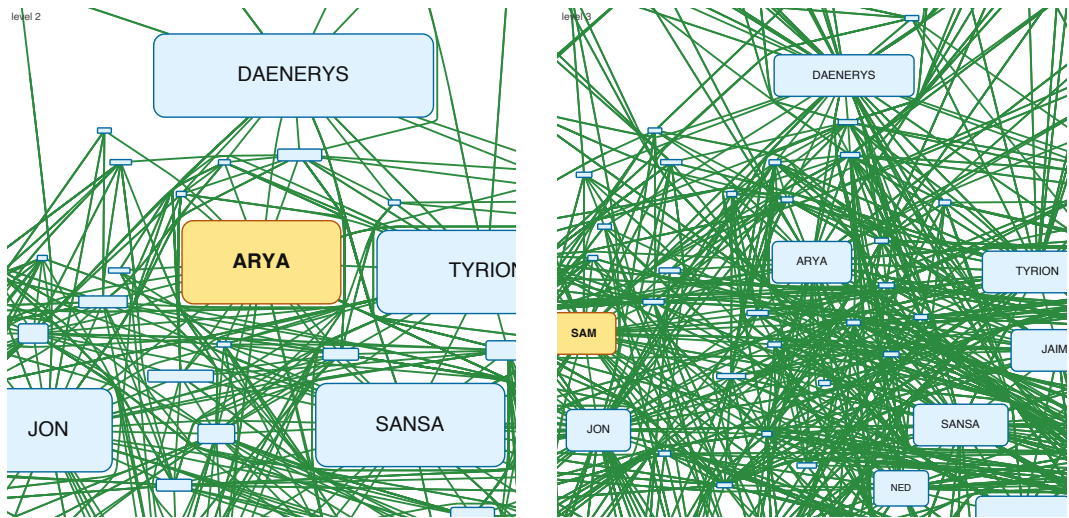


Figure 1 Overview of the per-level-graph tiling scheme presented in Section 4, on the Game of Thrones graph. Two adjacent pyramid levels are shown, cropped to the same window; the coarser level is on the left, the finer one on the right. For each level a new graph is assembled from a PageRank-ranked subset of the nodes, and the top-ranked landmark, highlighted in amber, enters that level enlarged by a factor of two in each dimension. Remaining survivors, shown in light blue, are kept at a node-specific scale $s_v \in [0.25, 2]$ that shrinks with rank so that they do not overlap. Edges are rerouted per level with a sleeve-hint A* built from the finer-level route, which keeps the coarse shape close to the fine one and yields smooth transitions as the user pans and zooms.

package, giving smooth pan and zoom. Live demos are available online: a WebGL renderer for large graphs⁵ and an SVG renderer for smaller, editable graphs⁶.

This paper makes two main contributions:

1. A *tiling scheme* for large graph visualization that builds a hierarchical tile pyramid, limits visible entities per tile using PageRank-based filtering, simplifies edge routes at coarser levels, and integrates with the WebGL renderer for real-time pan and zoom.
2. A *sleeve routing* algorithm that routes edges on the dual graph of a Constrained Delaunay Triangulation, using per-source batched Dijkstra trees followed by the classical funnel algorithm to produce shortest-path geodesics in homotopy classes.

2 Related Work

2.0.0.1 Graph visualization tools.

Graphviz [25, 5] is a long-standing graph drawing tool supporting multiple layout algorithms, including the Sugiyama method and Scalable Force-Directed Placement [7]; it does not support tiling or WebGL rendering. Our library builds on the earlier MSAGL desktop layout engine [39] that did not have the features described here. On the web, Sigma.js [8] and Cytoscape.js [23] provide interactive graph rendering, but rely on straight-line or generic spline edges without obstacle-avoiding routing and without a pyramid of precomputed tiles. ReGraph [6] uses WebGL to render large graphs with straight-line edges and supports inter-

⁵ <https://microsoft.github.io/msagljs/renderer-webgl/index.html>

⁶ <https://microsoft.github.io/msagljs/renderer-svg/index.html>

active cluster expansion but not tiling. Cosmograph [1] uses GPU-accelerated force-directed layout for million-node graphs with straight-line edges, without tiling. GraphMaps [38] supports tiling and level-of-detail for large graphs but runs only on Windows and does not optimize edge routes. Cornac [41] handles huge graphs with tiles and edge bundling [34, 32] but requires a multi-server backend. Earlier navigation schemes for large graphs include ASK-GraphView [12], which organizes nodes into a hierarchical cluster tree, and topological fisheye views [24], which distort a multilevel layout [27, 33] under the focus; neither targets browser-side tile pyramids or per-level edge rerouting. Hierarchical edge bundling [31] and force-directed bundling [32] are complementary clutter-reduction techniques applied *after* edges have been routed.

2.0.0.2 Shortest paths and edge routing.

Our sleeve algorithm walks the dual of a Constrained Delaunay Triangulation [43] and relies on the classical funnel algorithm for extracting a geodesic from a triangle strip [36, 15, 26, 4]. The optimal algorithm for Euclidean shortest paths among polygonal obstacles [30] and the recent $O(m \log^{2/3} n)$ single-source shortest-path algorithm that breaks the sorting barrier [20] are remarkable theoretical results, but they are not practical: both build elaborate global data structures that we avoid. Graphviz builds the full visibility graph for edge routing in force-directed layouts, which has $O(n^2)$ edges. For Sugiyama layouts, it uses the funnel algorithm [18] but requires a simple containing polygon. yWorks [11] offers “Organic edge routing” based on force-directed simulation. The approach of Dwyer and Nachmanson [22] routes on a Yao-graph spanner with local shortcutting. For orthogonal layouts, the libavoid framework of Wybrow, Marriott and Stuckey performs obstacle-avoiding connector routing with incremental updates [45]. Our sleeve method eliminates the spanner entirely and routes directly on the CDT dual graph, and is, to our knowledge, the first routing algorithm integrated with a browser-side tile pyramid that reroutes edges per level.

2.0.0.3 CDT-based pathfinding.

The closest precedent to our routing pipeline is the CDT-and-funnel pathfinder of Demyen and Buro [17], originally designed for game-AI units of nonzero radius. They run admissible A* search on the CDT dual (with a reduced-graph variant that collapses corridor chains into single abstract edges) to recover provably shortest paths. Their search is, however, per-query: the bounds depend on the start and goal, so it cannot share work across the many edges leaving a common source as our per-source Dijkstra does. For the graphs we target this would be roughly an order of magnitude slower; we therefore trade their proven optimality for batched throughput suitable for browser-side rerouting at every zoom level.

3 Loading a Graph

The loading of a graph with the WebGL renderer starts with the initial layout and routing is done for the whole graph, and then the tiling process. For this method to be visually stable we use IPsep-CoLa layout followed by sleeve routing, see Section 5.

1. **Layout.** Node positions are computed by the chosen algorithm. For large undirected graphs the default is force-directed layout, IPsepCola, initialized with Pivot MDS [14]; for directed graphs, Sugiyama layout is used.
2. **Edge routing.** Edges are routed around node obstacles. The routing mode is configurable; this paper focuses on *sleeve routing*, Section 5.

101 **3. Tiling.** A tile hierarchy is built for level-of-detail rendering, Section 4.

102 Next the renderer described in Section 6 consumes the tile hierarchy and renders the
103 appropriate level of detail based on the current viewport.

104 **4 Tiling**

105 To visualize large graphs interactively, MSAGLJS builds a pyramid of tiles analogous to those
106 used in online maps, served to the browser through the `TileLayer` of the `@deck.gl/geo-layers`
107 package [10]. Levels are numbered $0, 1, \dots, Z$ from coarsest to finest; level 0 is a single tile
108 covering the whole drawing with only a few top-ranking graph features, and the finest level Z
109 corresponds to the full graph produced by the layout and the sleeve router of Section 5.

110 Conceptually, each tile is associated with a rectangular viewport and holds the graph
111 elements that lie within it: the single tile at level 0 is associated with a viewport covering the
112 entire drawing, so it contains the whole graph. When rendering, every level shows a subset
113 of a rank-ordered prefix of the graph’s nodes: candidates are processed in order of decreasing
114 PageRank, and one is accepted only if it can be drawn at its natural size or larger without
115 its padded bounding box overlapping any already-accepted node; otherwise it is skipped.
116 Edges are inherited: an edge is shown only when both endpoints are accepted. Each tile then
117 renders the part of the accepted set that lies within its viewport; the remaining elements
118 are hidden. The per-tile capacity $\mathcal{C}$ only governs the choice of Z , that is, when to stop
119 subdividing, and is not consulted when deciding what to render at each level: the selection
120 of visible elements per level is driven entirely by importance and geometric non-overlap,
121 Section 4.1. Only the finest level Z displays every node and edge of the graph.

122 *Per-tile rendering load.* At the finest level, condition (i) is intended to cap each tile at
123 $\mathcal{C}$ elements, nodes plus edge clips, with default $\mathcal{C} = 500$. On dense graphs, however, either
124 the depth cap $Z_{\max} = 8$ or the minimum tile size (ii) binds first, in which case the densest
125 finest-level tile may hold many more than $\mathcal{C}$ elements. For a coarser level $z < Z$ we ignore
126 $\mathcal{C}$ when preparing a tile for rendering. We apply semantic zoom on the nodes, as detailed
127 in Section 4.1, by inflating each surviving node by roughly 2^{Z-z} in linear size. The chosen
128 nodes define the rendered edges: we render the edges connecting two chosen nodes. Table 1
129 reports the empirical worst-case per-tile element count on a range of graphs.

130 **4.0.0.1 How Z is chosen.**

131 Z is determined incrementally rather than fixed in advance. We initialize $Z = 0$ with the single
132 root tile covering the whole drawing. While the tiles at level Z violate conditions (i) or (ii)
133 below, we increase Z by one and split every tile of the previous finest level into four equal
134 sub-tiles. The two conditions are: (i) every tile contains at most $\mathcal{C}$ elements, with default
135 $\mathcal{C} = 500$, preventing overloaded tiles; and (ii) the tile width and height are both below ten
136 times the average node width and height, preventing tiles smaller than the nodes they display.
137 We additionally enforce a hard cap $Z \leq Z_{\max}$ derived from a memory estimate. At level z
138 the pyramid has a $2^z \times 2^z$ grid of tiles, so the finest grid resolution is $K = 2^Z$. Across the
139 whole pyramid we estimate the total number of stored tile elements as

$$140 \quad T(K) \approx 2 \left((K/4) M + N \right),$$

141 where N and M are the node and edge counts of G . The term $(K/4) M$ counts per-tile edge
142 clips at the finest level, under the heuristic that an average edge crosses $K/4$ tiles. The
143 additive N counts node copies. The factor 2 accounts for the contribution of all coarser

158 **Algorithm 1** BUILD_TILE_PYRAMID($G, \mathcal{C}, \text{minTileSize}$)

```

159 1:  $T_0 \leftarrow$  single tile holding all of  $G$  clipped to its bounding box
160 2:  $\text{levels} \leftarrow [T_0]$ ;  $z \leftarrow 0$ 
161 3: while some tile in  $\text{levels}[z]$  exceeds  $\mathcal{C}$  elements and its width/height  $\geq \text{minTileSize}$  do
162 4:    $\text{levels}[z+1] \leftarrow$  split every tile of  $\text{levels}[z]$  into four sub-tiles
163 5:   within each parent, distribute nodes/labels/arrowheads by bbox, and split each edge
164   clip at the tile midlines (Section 4.3)
165 6:    $z \leftarrow z + 1$ 
166 7: end while
167 8:  $Z \leftarrow z$ ;  $V_Z \leftarrow V(G)$ ;  $s_v^{(Z)} \leftarrow 1$  for all  $v \in V_Z$ 
168 9: compute PageRank( $G$ )
169 10: for  $z = Z - 1$  down to 0 do
170 11:    $(V_z, \{s_v^{(z)}\}) \leftarrow \text{SELECT\_TOP\_K\_WITH\_ADAPTIVE\_SCALE}(V_{z+1}, 2^{Z-z})$   $\triangleright$  Section 4.1
171 12:   build a CDT from the inflated polylines of  $V_z$  at scales  $\{s_v^{(z)}\}$ 
172 13:   reroute every edge  $e \in E_z$  on this CDT, using the level- $(z+1)$  route as a sleeve hint  $\triangleright$ 
173   Section 4.2
174 14:   for every tile  $t$  at level  $z$ , recompute the clips of  $e$  inside  $t$  from the new curve, splitting
175   at tile midlines
176 15: end for
177 16: return  $\text{levels}$ 

```

144 levels combined. With roughly 200 bytes per stored element and a maximum memory $\mathcal{M}_{\max}$
145 (default 4 GB), we solve $200 T(K) \leq \mathcal{M}_{\max}$ for the largest admissible K ,

$$146 \quad K_{\max} = \left\lfloor 4(\mathcal{M}_{\max}/(2 \cdot 200) - N)/M \right\rfloor,$$

147 and set

$$148 \quad Z_{\max} = \min(8, \lfloor \log_2 K_{\max} \rfloor).$$

149 The constant 8 is a safety ceiling: even when $\mathcal{M}_{\max}$ allows a much larger grid, we never
150 build more than nine pyramid levels.

151 4.0.0.2 Three phases.

152 Once Z and the tile rectangles are fixed, construction proceeds in three phases for every
153 coarser level $z = Z-1, Z-2, \dots, 0$: (1) use PageRank to pick the nodes shown at level z and
154 possibly enlarge them, Section 4.1; (2) reroute the edges of G_z around these obstacles, which
155 may now be enlarged, Section 4.2; (3) recompute the per-tile edge clips at level z by splitting
156 the new curves at tile boundaries, Section 4.3. Figure 1 on the first page shows two adjacent
157 pyramid levels built by this procedure. Algorithm 1 summarizes the whole pipeline.

178 4.1 PageRank-Guided Level Construction

179 To enable semantic zoom and keep the labels readable at coarse zooms, each level $0 \leq z < Z$
180 is assigned its own *level graph* $G_z = (V_z, E_z)$: a node subset $V_z \subseteq V$, together with every
181 edge of G whose endpoints both lie in V_z . We compute PageRank [40] on G once, sort all
182 nodes by descending PageRank as $v_0, v_1, \dots, v_{|V|-1}$. We write $S_z = 2^{Z-z}$ for the per-level
183 scale *ceiling*.

184 **4.1.0.1 Building V_z .**185 The candidate set at level z is the rank-prefix

186
$$T_z = \{v_0, v_1, \dots, v_{n_z-1}\}, \quad n_z = \lceil |V|/2^{Z-z} \rceil,$$

187 a fixed-size prefix of the globally rank-sorted node list, halving with each coarser level. We
 188 process T_z in rank-descending order. Each candidate v is assigned the largest scale $s_v \leq S_z$ at
 189 which its padded bounding box, scaled about v 's center, stays disjoint from the padded boxes
 190 of all already-accepted nodes; the padding absorbs the sleeve router's obstacle inflation and
 191 leaves a visible gap after triangulation. Disjointness is an axis-aligned separating-axis test,
 192 so the largest admissible scale $s^*(v)$ is obtained in closed form. If $s^*(v) \geq 1$ the candidate
 193 is accepted at $s_v = \min(\sigma, s^*(v))$, where the running cap σ , initially S_z , is then tightened
 194 to s_v so that no later, lower-ranked node is drawn larger than an already-accepted senior;
 195 otherwise the candidate is dropped. The accepted set is V_z and the per-node scales $\{s_v^{(z)}\}$ are
 196 stored. The result is a map-like effect: a few large landmarks at coarse zooms, progressively
 197 more detail as the user zooms in.

198 To avoid testing every candidate against every previously accepted box, accepted boxes
 199 are maintained in a uniform spatial hash whose cell size is an upper bound on the inflated
 200 box dimension at the current level. Two boxes can overlap only if their center cells are within
 201 one step on each axis, so each candidate consults at most nine cells. On the layouts we tested
 202 the expected work per candidate is $O(1)$, giving an overall expected cost of $O(|T_z| \log |T_z|)$
 203 dominated by the initial sort; the worst case, when many boxes pile into a single cell, degrades
 204 to $O(|T_z| \cdot |V_z|)$. In our experiments this phase was never a bottleneck: the per-level rerouting
 205 of Section 4.2 dominates the build time at every level we measured.

206 **4.2 Stable Per-Level Rerouting with a Routing Hint**

207 At each level we change the graph and the node sizes, so the edges must be rerouted on each
 208 level as well. We rerun the sleeve router, Section 5, on the set of the level edges.

209 **4.2.0.1 Per-level CDT.**

210 For level $z-1$ we build a dedicated CDT from the inflated polylines of V_{z-1} only, using
 211 each node's level- $z-1$ scale. The global landmark v_0 , for example, enters the triangulation as
 212 an obstacle that is 2^{Z-z+1} times its finest-level size in each dimension, so routes naturally bend
 213 around it.

214 **4.2.0.2 Routing hint.**

215 For every edge $e \in E_{z-1}$ we already have a route c_e computed at the finer level z . Sampling c_e
 216 and listing the level- $z-1$ CDT triangles it visits, expanded by one neighbor hop, yields a
 217 triangle *sleeve* S_e that we can use as a per-edge hint for an A* search restricted to S_e . This
 218 keeps the coarse route close to the fine one, so an edge deforms smoothly as the user zooms
 219 out instead of flipping to the other side of a landmark. The price is one independent A* per
 220 edge.

221 By default we instead reuse the per-source Dijkstra trees of Section 5.2, which are faster
 222 and behave well in practice. The reason this option is not always available is that, in a
 223 subgraph, the set of transparent obstacles is per-edge rather than per-source: the boundaries
 224 of nodes on the ancestor chains of the edge's endpoints may be crossed, and for edges sharing
 225 a source these chains differ with the target. A single Dijkstra tree from the source therefore

Table 1 Maximum number of elements rendered in a single tile across the whole pyramid for several graphs (capacity $\mathcal{C} = 500$, $Z_{\max} = 8$). The capacity $\mathcal{C}$ only guides the choice of Z ; we have no tight analytical bound on the actual rendered per-tile element count, which is therefore reported empirically. “Max tile’s level” is the level on which the maximum tile was found. In the worst cases (**facebook_combined**, **ca-CondMat**) the densest tile holds 20–30 $\times$ the nominal capacity $\mathcal{C}$.

Graph	$ V $	$ E $	Levels built	Max per tile	Max tile’s level
gameofthrones	407	2 639	5	1 168	4
composers	3 405	13 832	8	3 317	7
ca-GrQc	5 241	14 484	8	934	7
ca-HepTh	9 875	25 973	9	1 598	8
facebook_combined	4 039	88 234	9	11 874	8
ca-CondMat	23 133	186 878	9	13 744	8
deezer_europe	28 281	92 752	9	8 201	8
delaunay_n15	32 768	98 274	9	611	8

cannot serve all of its outgoing edges in this setting, so the hint+A* variant is used in this case. The graphs listed in the paper do not have subgraphs.

4.3 Splitting Edges between Tiles

Once each level G_z has its final geometry, we cut that level into tiles. A tile is a pair $(rect, tiledata)$, where *tiledata* is a set of *tile elements*: nodes, edge labels, edge arrowheads, and *edge clips*. An edge clip is a pair (e, p) , with p a continuous piece of the edge curve c_e confined to *rect*.

The single tile at level 0 is obtained by clipping every element to the graph’s bounding box. To build level $z + 1$ from level z , each tile is split into four equal sub-tiles; nodes, labels, and arrowheads are assigned to sub-tiles by bounding-box intersection, and each edge clip is cut at the tile midlines into sub-clips confined to individual sub-tiles. We maintain invariant $\mathcal{F}$: (a) every clip stays inside its tile’s rectangle, crossing the boundary only at its endpoints; (b) the union of all clips for edge e inside a tile equals $c_e \cap rect$. Part (a) makes subdivision cheap and local: thanks to (a) we never need to clip against the four outer sides of a tile, only against its two midlines. For the default polyline edges produced by the sleeve router this is trivial—each segment meets each midline at most once—so a clip is split by walking its segments and cutting at the midline crossings. The optional Bezier-corner pass of Section 5.3 replaces these line–line intersections with curve–line ones plus a parameter sort, but obeys the same invariant. Part (b) is the rendering-correctness condition: at any zoom only the tiles intersecting the viewport are fetched and their clips drawn, and (b) guarantees that the union of what is drawn equals $c_e \cap viewport$ for every visible edge e . The same invariant makes the rerouting of Section 4.2 compatible with tiling: whenever an edge curve is replaced by a coarser-level route, (b) is reestablished by recomputing the affected clips.

Subdivision of a level stops as soon as either of the following holds: (i) every tile at that level already contains at most $\mathcal{C}$ elements; or (ii) the tile width and height have both dropped below ten times the average node width and height, respectively. These are the same conditions that determine Z (Algorithm 1).

Table 1 reports the resulting maximum number of elements rendered in a single tile across the entire pyramid for a range of graphs.

269 **5 Sleeve Routing on the CDT**

270 We describe an edge routing method that routes edges directly on the dual graph of a
 271 Constrained Delaunay Triangulation, without requiring a spanner graph. The method finds a
 272 *sleeve*—a sequence of adjacent CDT triangles from source to target—and applies the funnel
 273 algorithm to produce the shortest path within it.

274 **5.1 CDT Construction and the Dual Graph**

275 Given the graph layout, each node is covered by a padded obstacle polygon. We compute a
 276 Constrained Delaunay Triangulation $\mathcal{T}$ on the set $\mathcal{O}$ of these obstacle polygons [16, 19]. The
 277 *dual graph* D of $\mathcal{T}$ has one node per triangle and one edge for each pair of triangles sharing
 278 a side. A triangle is *free-space* if it is not contained in any obstacle interior.

279 **5.2 Routing inside a Sleeve**

280 Intuitively, the sleeve is a chain of triangles meeting along shared edges, the *diagonals*, that
 281 the path must cross in order. Routing an edge from s to t splits into two subproblems:
 282 a *combinatorial* one—which sequence of diagonals to cross—and a *geometric* one—which
 283 exact polyline through that sequence is shortest. The geometric step is the classical funnel
 284 algorithm [15, 29], which pulls the path taut inside the polygon formed by the sleeve triangles.
 285 The sleeve router itself is therefore agnostic to how the sleeve is produced; any search on the
 286 dual graph that returns such a sequence of triangles will do.

287 To obtain the sleeve we run a shortest-path search on D where the step from one triangle
 288 to a neighbor costs the Euclidean distance between their centroids; triangles inside obstacles
 289 other than the source and target obstacles are excluded. The natural per-edge instantiation
 290 is A^* [28] with the straight-line distance to t as heuristic. When a node has many incident
 291 edges, however, running independent searches is wasteful. We instead group edges by source s
 292 and run a single Dijkstra computation per source, expanding until all targets are reached
 293 and recovering each sleeve by following parent pointers. This reduces the number of searches
 294 from m to at most n , the number of nodes; on the Game of Thrones social network (407
 295 nodes, 2 639 edges) it cuts routing time by 30%.

296 Since the query edges are undirected on D , each can be served from either endpoint,
 297 so the number of distinct roots is in turn reducible by reorienting query edges to minimize
 298 that count—an instance of minimum vertex cover on the demand graph, well approximated
 299 in linear time by the greedy maximum-degree rule and used as a source-side speed-up in
 300 many-to-many shortest-path frameworks [35]. Applying this heuristic reduces the number of
 301 Dijkstra trees from 331 (one per distinct source) to 199 on Game of Thrones and from 2 645
 302 to 1 285 on the composers graph (3 405 nodes, 13 832 edges), shortening the routing pass by
 303 $\sim 17\%$ and $\sim 35\%$ respectively.

304 Before launching the sleeve search for an edge (s, t) we attempt an even cheaper shortcut:
 305 starting at the triangle containing s , we walk the triangulation along the segment $\overline{st}$ triangle
 306 by triangle and check whether any traversed triangle is interior to a third obstacle. If none
 307 is, the straight segment is the geodesic and we use it directly, skipping both the dual-graph
 308 search and the funnel.

309 **5.3 Vertex Collapse at Source and Target**

310 We improve the sleeve’s geometry in the situation depicted in Figure 2. The remedy is to
 311 *collapse* corner-hugging chain vertices to the corresponding endpoint. Walking the right

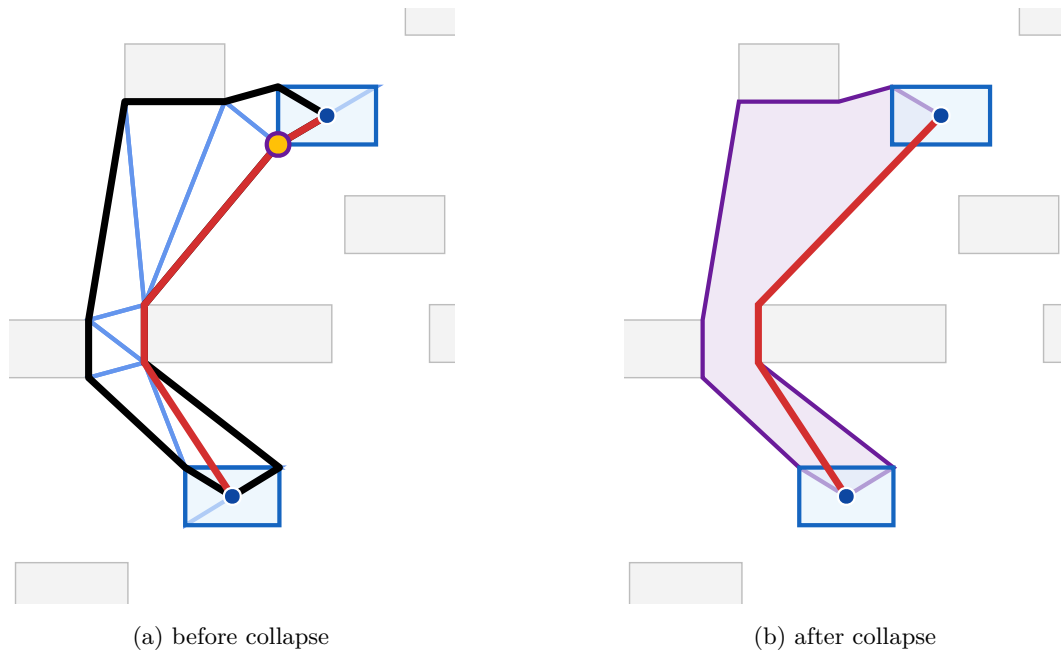


Figure 2 Effect of vertex collapse on the LORAS-RENLY edge of the Game of Thrones graph. Endpoints are shown as blue dots; their padded obstacles are highlighted in light blue. (a) The CDT triangles forming the sleeve are drawn in cornflower blue and the funnel path in red; the thick black polyline is the perimeter of the non-collapsed sleeve. The yellow dot marks the source-obstacle vertex that gets collapsed. (b) The sleeve after collapse, drawn as a single purple polygon with the source/target obstacles merged into the corresponding endpoints, and the funnel path (red) through it. Collapsing widens the funnel opening at the endpoint and removes the spurious detour around the obstacle corner.

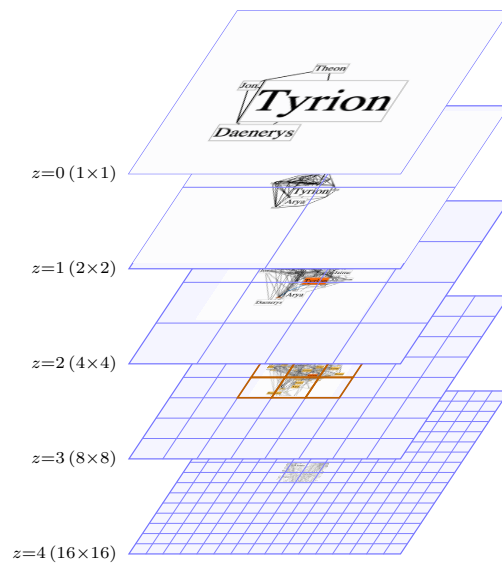
chain of the sleeve outward from s , we find the first vertex of the source obstacle at which the chain turns right toward s , and replace every source-owned vertex on that chain up to and including this one by s . The left chain is processed mirror-symmetrically (with left turns instead of right), and the target end likewise. Diagonals whose two endpoints coincide after collapse are dropped, and the left/right orientation of each surviving diagonal is preserved from the uncollapsed geometry. The collapsed vertex in Figure 2(a) is marked in yellow.

After collapse, the funnel sees a wide opening at each endpoint, as in Figure 2(b), and naturally selects the optimal exit direction. A final arrowhead-trimming step clips the path at the node boundary curves. Collapse might cause the resulting path to overlap nodes other than the source and the target, but we have found the effect to be negligible.

The implementation also offers an optional pass that fits a Bezier segment into each polyline corner to render edges as smooth curves. It is disabled by default for performance: the polyline rendering is fast and visually acceptable, so we trade the smoother curves for routing speed.

6 WebGL Rendering

The WebGL renderer uses deck.gl [2], a GPU-accelerated framework for large-scale data visualization. The renderer sets up an `OrthographicView`, with no perspective distortion, and a `TileLayer` that dynamically loads tile data based on the current viewport. Because



354 ■ **Figure 3** The entities rendered on each level as the user zooms in.

338 deck.gl's **TileLayer** expects a power-of-two pyramid of 512-pixel tiles, the root tile size is
 339 rounded up to the nearest power of two and a *start zoom* level is computed so that the
 340 root tile maps to deck.gl's 512-pixel tile size. The **TileLayer**'s `getTileData` callback then
 341 translates deck.gl tile coordinates (x, y, z) to MSAGLJS tile map coordinates, fetching the
 342 precomputed tile data.

343 In deck.gl's **OrthographicView** the screen scale at view zoom v is $s = 2^v$ pixels per world
 344 unit, and the start zoom is $v_0 = \log_2(512/W)$, so that at $v = v_0$ the root tile of width W
 345 projects to 512 pixels. The **TileLayer** then requests pyramid level $z = \lfloor v - v_0 \rfloor$, at which
 346 each tile covers $W/2^z$ world units and renders at about 512 pixels. Note v_0 lives in the same
 347 view-zoom space as v and is not a pyramid level: when the bounding box is bigger than
 348 one 512-pixel tile, v_0 is negative. For example, with a bounding box of 1000×800 world
 349 units rounded up to $W = 1024$ we get $v_0 = -1$; at $v = 2$ the scale is $s = 4$ px/wu, the
 350 chosen level is $z = 3$, and each level-3 tile spans 128 wu. A 1280×800 -pixel viewport then
 351 covers 320×200 wu and intersects $\lceil 320/128 \rceil \times \lceil 200/128 \rceil = 3 \times 2 = 6$ level-3 tiles, the only
 352 ones fetched (Figure 3). Between integer view zooms the level is held fixed and the tiles are
 353 GPU-scaled, so continuous panning and zooming never block on routing or rerouting.

355 6.0.0.1 Rendering layers.

356 Each tile is rendered as a composite deck.gl layer containing:

- 357 ■ A **GeometryLayer** for node shapes (rectangles, ovals, diamonds).
- 358 ■ A **TextLayer** for node labels.
- 359 ■ A custom **CurveLayer** for edge paths, supporting cubic Bézier curves, arcs, and line
 360 segments via GPU-side interpolation. Curve tessellation resolution scales with zoom: at
 361 zoom level z , the resolution is 2^{z-2} segments per curve span.
- 362 ■ An **IconLayer** for edge arrowheads.
- 363 ■ A **TextLayer** for edge labels.

Graph	$ V $	$ E $	Routing	Tiling	Total
gameofthrones	407	2 639	0.17	0.22	1.94
composers	3 405	13 832	4.37	5.20	10.39
ca-GrQc	5 241	14 484	4.01	3.15	8.42
ca-HepTh	9 875	25 973	26.83	15.95	46.07
ca-HepPh	12 008	236 978	82.10	156.60	246.30
facebook_combined	4 039	88 234	12.25	29.87	43.82
ca-CondMat	23 133	186 878	224.20	188.10	431.20
deezer_europe	28 281	92 752	384.20	143.00	552.20
delaunay_n15	32 768	98 274	31.95	16.08	76.30

Table 2 End-to-end loading benchmarks on an Apple M4 Max laptop, all times in seconds. *Routing* sums the CDT-construction and Dijkstra-tree sleeve-search phases of Section 5.2 on the full graph; *Tiling* is the build of the tile pyramid (capacity $C = 500$, $Z_{\max} = 8$) of Section 4, which includes the per-level rerouting; *Total* adds parsing.

6.0.0.2 Interaction.

The renderer supports pan, zoom, hover highlighting, and click selection. Hovering an edge highlights it and its incident nodes. The `zoomTo` API animates the viewport to a target rectangle with smooth transitions.

7 Experimental Results

We evaluate the sleeve router and the tile-pyramid build on the same benchmark graphs as Table 1: the Game of Thrones social network [13], the GD 2011 contest `composers` graph [9], four SNAP collaboration networks (`ca-GrQc`, `ca-HepTh`, `ca-HepPh`, `ca-CondMat`) and the SNAP `facebook_combined` ego network [37, 3], the `deezer_europe` social network [42], and the SuiteSparse Delaunay mesh `delaunay_n15`. All experiments run in Node.js (single process, headless; not a browser) on an Apple M4 Max laptop with 48 GB of unified memory. Layout uses IPsepCola, and the tile pyramid is built with the same parameters as Table 1, capacity $C = 500$ and $Z_{\max} = 8$.

Table 2 reports per-graph timings of three pipeline stages: *Routing*, the sum of the two phases logged by the sleeve router on the full graph—constrained Delaunay triangulation of the obstacles and the Dijkstra-tree search on the CDT dual (Section 5.2); *Tiling*, the build of the tile pyramid (Section 4), which includes the per-level rerouting of Section 4.2; and *Total*, the end-to-end loading time including parsing.

The CDT phase contributes negligibly to the routing column—from 8 ms on Game of Thrones to about half a second on the largest graphs—so virtually all of *Routing* is the Dijkstra-tree sleeve search, with the vertex-cover heuristic of Section 5.2 reducing the number of trees we run. Even on graphs with hundreds of thousands of edges (`ca-CondMat`, `ca-HepPh`, `deezer_europe`) the full pipeline finishes in a few minutes; this is a one-time build cost. The resulting tile pyramid already contains the per-level routes of Section 4.2, so panning and zooming only fetch and render the precomputed tiles, with no rerouting at view time.

8 Conclusion

We presented MSAGLJS, an open-source TypeScript library for graph layout, edge routing, and visualization in web browsers. The sleeve routing method routes edges directly on the CDT dual graph using batched Dijkstra trees and the funnel algorithm. The tiling scheme enables map-like exploration of large graphs with level-of-detail filtering and edge

simplification. The WebGL renderer, built on deck.gl, provides smooth pan-and-zoom interaction.

MSAGLJS is available at <https://github.com/microsoft/msagljs> under the MIT license.

References

- 1 Cosmograph. <https://cosmograph.app>.
- 2 deck.gl: Large-scale WebGL-powered data visualization. <https://deck.gl/>.
- 3 facebookcombined. https://snap.stanford.edu/data/facebook_combined.txt.gz.
- 4 Funnel algorithm. <https://page.mi.fu-berlin.de/mulzer/notes/alggeo/polySP.pdf>.
- 5 Graphviz. <http://www.graphviz.org/>.
- 6 Regraph. <https://cambridge-intelligence.com/regraph/>.
- 7 sfdp. <https://graphviz.org/docs/layouts/sfdp/>.
- 8 Sigma.js: a JavaScript library aimed at visualizing graphs of thousands of nodes and edges. <https://www.sigmajs.org/>.
- 9 Skewed. <http://mozart.diei.unipg.it/gdcontest/contest2011/composers.xml>.
- 10 TileLayer (@deck.gl/geo-layers). <https://deck.gl/docs/api-reference/geo-layers/tile-layer>. Accessed: 2026.
- 11 yworks. <https://yworks.com/products/yed>.
- 12 James Abello, Frank Van Ham, and Neeraj Krishnan. ASK-GraphView: A large scale graph visualization system. In *IEEE Transactions on Visualization and Computer Graphics*, volume 12, pages 669–676. IEEE, 2006.
- 13 Andrew Beveridge and Michael Chemers. The game of game of thrones: Networked concordances and fractal dramaturgy. In *Reading Contemporary Serial Television Universes*, pages 201–225. Routledge, 2018.
- 14 Ulrik Brandes and Christian Pich. Eigensolver methods for progressive multidimensional scaling of large data. In *Graph Drawing: 14th International Symposium, GD 2006, Karlsruhe, Germany, September 18–20, 2006. Revised Papers 14*, pages 42–53. Springer, 2007.
- 15 Bernard Chazelle. A theorem on polygon cutting with applications. In *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, pages 339–349. IEEE, 1982.
- 16 Boris Delaunay et al. Sur la sphere vide. *Izv. Akad. Nauk SSSR, Otdelenie Matematicheskii i Estestvennyka Nauk*, 7(1):793–800, 1934.
- 17 Douglas Demyen and Michael Buro. Efficient triangulation-based pathfinding. In *Proceedings of the 21st National Conference on Artificial Intelligence (AAAI)*, pages 942–947, 2006.
- 18 David P Dobkin, Emden R Gansner, Eleftherios Koutsofios, and Stephen C North. Implementing a general-purpose edge router. In *Graph Drawing: 5th International Symposium, GD'97 Rome, Italy, September 18–20, 1997 Proceedings 5*, pages 262–271. Springer, 1997.
- 19 Vid Domiter and Borut Žalik. Sweep-line algorithm for constrained delaunay triangulation. *International Journal of Geographical Information Science*, 22(4):449–462, 2008.
- 20 Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu, and Longhui Yin. Breaking the sorting barrier for directed single-source shortest paths. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, 2025. <https://arxiv.org/abs/2504.17033>.
- 21 Tim Dwyer, Yehuda Koren, and Kim Marriott. IPSep-CoLa: An incremental procedure for separation constraint layout of graphs. *IEEE Transactions on Visualization and Computer Graphics*, 12(5):821–828, 2006. doi:10.1109/TVCG.2006.156.
- 22 Tim Dwyer and Lev Nachmanson. Fast edge-routing for large graphs. In *Graph Drawing: 17th International Symposium, GD 2009, Chicago, IL, USA, September 22–25, 2009. Revised Papers 17*, pages 147–158. Springer, 2010.
- 23 Max Franz, Christian T. Lopes, Gerardo Huck, Yue Dong, Onur Sumer, and Gary D. Bader. Cytoscape.js: a graph theory library for visualisation and analysis. *Bioinformatics*, 32(2):309–311, 2016.

- 458 **24** Emden R. Gansner, Yehuda Koren, and Stephen North. Topological fisheye views for visualizing
459 large graphs. *IEEE Transactions on Visualization and Computer Graphics*, 11(4):457–468,
460 2005.
- 461 **25** Emden R. Gansner and Stephen C. North. An open graph visualization system and its
462 applications to software engineering. *Software: Practice and Experience*, 30(11):1203–1233,
463 2000.
- 464 **26** Leonidas Guibas, John Hershberger, Daniel Leven, Micha Sharir, and Robert E. Tarjan.
465 Linear-time algorithms for visibility and shortest path problems inside triangulated simple
466 polygons. *Algorithmica*, 2(1–4):209–233, 1987.
- 467 **27** David Harel and Yehuda Koren. A fast multi-scale method for drawing large graphs. In
468 *Journal of Graph Algorithms and Applications*, volume 6, pages 179–202, 2002.
- 469 **28** Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic
470 determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*,
471 4(2):100–107, 1968. doi:10.1109/TSSC.1968.300136.
- 472 **29** John Hershberger and Jack Snoeyink. Computing minimum length paths of a given homotopy
473 class. *Computational geometry*, 4(2):63–97, 1994.
- 474 **30** John Hershberger and Subhash Suri. An optimal algorithm for Euclidean shortest paths in
475 the plane. *SIAM Journal on Computing*, 28(6):2215–2256, 1999.
- 476 **31** Danny Holten. Hierarchical edge bundles: Visualization of adjacency relations in hierarchical
477 data. *IEEE Transactions on Visualization and Computer Graphics*, 12(5):741–748, 2006.
- 478 **32** Danny Holten and Jarke J. Van Wijk. Force-directed edge bundling for graph visualization.
479 In *Computer Graphics Forum*, volume 28, pages 983–990. Wiley Online Library, 2009.
- 480 **33** Yifan Hu. Efficient, high-quality force-directed graph drawing. *Mathematica Journal*, 10(1):37–
481 71, 2005.
- 482 **34** Christophe Hurter, Ozan Ersoy, and Alexandru Telea. Graph bundling by kernel density
483 estimation. In *Computer graphics forum*, volume 31, pages 865–874. Wiley Online Library,
484 2012.
- 485 **35** Sebastian Knopp, Peter Sanders, Dominik Schultes, Falk Schieferdecker, and Dorothea Wagner.
486 Computing many-to-many shortest paths using highway hierarchies. In *Proceedings of the 9th*
487 *Workshop on Algorithm Engineering and Experiments (ALENEX)*, pages 36–45. SIAM, 2007.
- 488 **36** D. T. Lee and Franco P. Preparata. Euclidean shortest paths in the presence of rectilinear
489 barriers. *Networks*, 14(3):393–410, 1984.
- 490 **37** Jure Leskovec, Jon Kleinberg, and Christos Faloutsos. Graph evolution: Densification and
491 shrinking diameters. *ACM transactions on Knowledge Discovery from Data (TKDD)*, 1(1):2–es,
492 2007.
- 493 **38** Lev Nachmanson, Roman Prutkin, Bongshin Lee, Nathalie Henry Riche, Alexander E Holroyd,
494 and Xiaoji Chen. Graphmaps: Browsing large graphs as interactive maps. In *Graph Drawing*
495 *and Network Visualization: 23rd International Symposium, GD 2015, Los Angeles, CA, USA,*
496 *September 24–26, 2015, Revised Selected Papers 23*, pages 3–15. Springer, 2015.
- 497 **39** Lev Nachmanson, George G. Robertson, and Bongshin Lee. Drawing graphs with GLEE. In
498 *Graph Drawing: 15th International Symposium, GD 2007*, pages 389–394. Springer, 2008.
- 499 **40** Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation
500 ranking: Bringing order to the web. *Stanford InfoLab*, 249(373):1–17, 1999.
- 501 **41** Alexandre Perrot and David Auber. Cornac: Tackling huge graph visualization with big data
502 infrastructure. *IEEE Transactions on Big Data*, 6(1):80–92, 2018.
- 503 **42** Benedek Rozemberczki and Rik Sarkar. Characteristic Functions on Graphs: Birds of a
504 Feather, from Statistical Descriptors to Parametric Models. In *Proceedings of the 29th ACM*
505 *International Conference on Information and Knowledge Management (CIKM '20)*, page
506 1325–1334. ACM, 2020.
- 507 **43** Jonathan Richard Shewchuk. Delaunay refinement algorithms for triangular mesh generation.
508 *Computational Geometry*, 22(1–3):21–74, 2002.

- 509 **44** Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiro Toda. Methods for visual understanding
510 of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*,
511 11(2):109–125, 1981. doi:10.1109/TSMC.1981.4308636.
- 512 **45** Michael Wybrow, Kim Marriott, and Peter J. Stuckey. Orthogonal connector routing. *Lecture*
513 *Notes in Computer Science*, 5849:219–231, 2010.